

ML Machine Learning (S2-25_AIMLCZG565) – Sem1 → Assignment 2

BITS ID - 2025ac05520

NAME – Nitin B. Kale

ML Assignment 2 - Classification Model Deployment

Assignment Given:

In this assignment, we are required to: - Implement multiple classification models – Build an interactive Streamlit web application to demonstrate your models
Deploy the app on Streamlit Community Cloud (FREE) - Share clickable links for evaluate - **DONE**

Mandatory Submission - Status

- 1- Links Each submission must be a single PDF file with the following- **DONE**
- 2- GitHub Repository Link containing **DONE**
 - a. Complete source code
 - b. requirements.txt
 - c. A clear README.md
 - d. Test data used in your experiments (csv)
- 3- Live Streamlit App Link: Deployed using Streamlit Community Cloud, Must open an interactive frontend when clicked **DONE**
- 4- Screenshot: Upload screenshot of assignment execution on BITS Virtual Lab **DONE**
- 5- The Github README content (details mentioned in Sec on 3 - Step 5) should also be part of the submitted PDF file **DONE**

Problem Statement:

Build and compare multiple classification models on a public dataset, then expose the results through an interactive Streamlit app that can evaluate uploaded test data.

Dataset Description:

- I. Dataset used: Breast Cancer Wisconsin (Diagnostic)
- II. Source: UCI Machine Learning Repository
- III. Type: Binary classification
- IV. Samples: 569

V. Features: 30 numeric features

Target classes:

- 0 = malignant
- 1 = benign

GitHub Repository Link: →

https://github.com/nitinkale14/ml_semister_assignment_2/

Live Streamlit App Link:→

<https://mlsemisterassignment2-bsucnevwhmnavkmxbugvqu.streamlit.app/>

Models and Execution Details:

The assignment text lists five explicit models and one line that says six. I implemented the five models that are actually enumerated in the table and instructions.

ML Model Name	Accuracy	AUC	Precision	Recall	F1	MCC
Logistic Regression	0.9825	0.9957	0.9861	0.9861	0.9861	0.9623
Decision Tree	0.9123	0.9157	0.9559	0.9028	0.9286	0.8174
KNN	0.9561	0.9788	0.9589	0.9722	0.9655	0.9054
Naive Bayes	0.9386	0.9878	0.9452	0.9583	0.9517	0.8676
Random Forest (Ensemble)	0.9561	0.9931	0.9589	0.9722	0.9655	0.9054

Observations

ML Model Name	Observation about model performance
Logistic Regression	Best overall on this split. It achieved the highest accuracy, AUC, and MCC, which suggests the dataset is largely linearly separable.
Decision Tree	The weakest generalizer here. It is easy to explain, but the single-tree structure loses stability compared with the other models.
KNN	Strong second-tier result after scaling. Its performance is close to the ensemble model on this split.
Naive Bayes	Competitive and fast, but still below the strongest models because of its independence assumption.
Random Forest (Ensemble)	Very strong AUC and F1, but it still trails logistic regression on the main ranking metrics for this split.

Overall winner for this dataset:

Logistic Regression as it achieved the highest accuracy, AUC, and MCC, which suggests the dataset is largely linearly separable.

Project Structure:

The screenshot displays a code editor interface with a project structure on the left and a code editor on the right. The project structure includes files like `metadata.json`, `metrics.csv`, `naive_bayes.joblib`, `random_forest.joblib`, `test`, `.gitignore`, `app.py`, `ml_utils.py`, `README.md`, `requirements.txt`, `test_data.csv`, and `train_models.py`. The code editor shows the `train_models.py` file with the following code:

```
1 from __future__ import annotations
2
3 > import ...
4
5
6
7
8
9
10 def main() -> None: 1 usage  & Nitin
11     metadata = train_and_export(random_state=42)
12     summary = pd.DataFrame.from_dict(metadata["metrics"], orient="index")
13     print("Saved model artifacts to:", Path(PROJECT_ROOT / "model").resolve())
14     print("Saved test data to:", Path(PROJECT_ROOT / "test_data.csv").resolve())
15     print()
16     print(summary.round(4).to_string())
17
18
19 if __name__ == "__main__":
20     main()
21
```

The Run panel at the bottom shows the execution output for `train_models`:

```
saved model artifacts to: C:\Users\Nitin\Downloads\ML_Assignment2\ML_Assignment2-main\model
Saved test data to: C:\Users\Nitin\Downloads\ML_Assignment2\ML_Assignment2-main\test_data.csv
```

	Accuracy	AUC	Precision	Recall	F1	MCC
Logistic Regression	0.9825	0.9957	0.9861	0.9861	0.9861	0.9623
Decision Tree	0.9123	0.9157	0.9559	0.9028	0.9286	0.8174
KNN	0.9561	0.9788	0.9589	0.9722	0.9655	0.9054
Naive Bayes	0.9386	0.9878	0.9452	0.9583	0.9517	0.8676
Random Forest	0.9561	0.9931	0.9589	0.9722	0.9655	0.9054

Running app.py:

The image shows a VS Code editor with the `app.py` file open. The code defines a function `evaluate_models_on_data` that takes a `data_frame`, `model_names`, `feature_names`, `target_column`, and `class_names` as input. It uses `pd.DataFrame` and `tuple` to process the data and return a tuple containing a list of `SummaryRow` objects and a dictionary of detailed results.

```
def evaluate_models_on_data(
    data_frame: pd.DataFrame,
    model_names: tuple[str, ...],
    feature_names: tuple[str, ...],
    target_column: str,
    class_names: tuple[str, ...],
) -> tuple[list[SummaryRow], dict[str, DetailedResult]]:
    cache_key = evaluation_cache_key(
        data_frame,
        model_names=model_names,
        feature_names=feature_names,
        target_column=target_column,
        class_names=class_names,
    )

    cached = EVALUATION_CACHE.get(cache_key)
    if cached is not None:
        return cached

    features = align_feature_frame(data_frame, list(feature_names))
    target = extract_target_series(data_frame, target_column)
    models = load_saved_models_cached(model_names)

    summary_rows: list[SummaryRow] = []
    detailed_results: dict[str, DetailedResult] = {}
    for model_name, model in models.items():
        result = compute_classification_metrics(
            model,
            features,
            target,
            class_names,
```

The Streamlit web application is running on `localhost:8501`. It features an "Input" section with an "Upload test CSV" button and a "Model" dropdown menu set to "Logistic Regression". The application displays a table of test data and a "Model comparison" table showing the performance of various models.

	5	13.17	18.22	64.28	537.3	0.0747
6	8.878	15.49	56.74	241	0.0829	
7	19.68	21.68	129.9	1194	0.098	
8	18.31	20.58	120.8	1052	0.1068	
9	20.6	29.33	140.1	1265	0.1178	

Model	Accuracy	AUC	Precision	Recall	F1	MCC
Logistic Regression	0.982	0.996	0.986	0.986	0.986	0.962
KNN	0.956	0.979	0.959	0.972	0.966	0.905
Random Forest	0.956	0.993	0.959	0.972	0.966	0.905
Naive Bayes	0.939	0.988	0.945	0.958	0.952	0.868
Decision Tree	0.912	0.916	0.956	0.903	0.929	0.817

Best model on this test data: Logistic Regression

Detailed view: Logistic Regression

Accuracy	AUC	Precision
98.25%	99.57%	98.61%

Lab Screenshot:

https://argo-rdp.codeargo.net/web-rdp/#/client/aS0wZDk4MWwNkNzg0ZDgwMzc4MABjAGpzb24?token=5F194A3057F47D72622ADC0434FBA9ADEBC1D6072BAB3C5555F8C16B38FE462D

Applications: ml_semester_assignme... Firefox Thunar Xfce Terminal

New Tab ML Assignment 2 x

localhost:8501

Deploy

Binary classification comparison for the Breast Cancer Wisconsin (Diagnostic) dataset.

Dataset preview

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean com
0	19.55	28.77	133.6	1207	0.0926	
1	11.13	16.62	70.47	381.1	0.0815	
2	13.82	24.49	92.33	595.9	0.1162	
3	16.5	18.29	106.6	838.1	0.0969	
4	21.56	22.39	142	1479	0.111	
5	13.17	18.22	84.28	537.3	0.0747	
6	8.878	15.49	56.74	241	0.0829	
7	19.68	21.68	129.9	1194	0.098	
8	18.31	20.58	120.8	1052	0.1068	
9	20.6	29.33	140.1	1265	0.1178	

Model comparison

Model	Accuracy	AUC	Precision	Recall	F1	MCC
Logistic Regression	0.982	0.996	0.986	0.986	0.986	0.962
KNN	0.956	0.979	0.959	0.972	0.966	0.905
Random Forest	0.956	0.993	0.959	0.972	0.966	0.905

Project ml_semester_assignme... Version control

ml_semester_assignme...

- .venv
- model
 - decision_tree
 - knn.joblib
 - logistic_regr
 - metadata.json
 - metrics.csv
 - naive_bayes
 - random_forest
- test
- app.py
- ml_utils.py

```
1 from __future__ import annotations
2
3 import subprocess
4 import sys
5 from pathlib import Path
6 from typing import Any, Mapping, cast
7
8 import matplotlib.pyplot as plt
9 import pandas as pd
10 import seaborn as sns
11 import streamlit as st
12 from streamlit.runtime.scriptrunner import get_script_run_ctx
13
14 from ml_utils import (
15     EVALUATION_CACHE,
16     PROJECT_ROOT,
17     align_feature_frame,
18     compute_classification_metrics,
19     evaluation_cache_key,
20     extract_target_series,
21     load_metadata_cached,
22     load_saved_models_cached,
23 )
24
25 SummaryRow = dict[str, Any]
26 DetailedResult = dict[str, Any]
27
```

Run app x

```
/home/cloud/Desktop/ml_semester_assignment_2-main/.venv/bin/python /home/cloud/Desktop/ml_semis
```

You can now view your Streamlit app in your browser.

Local URL: <http://localhost:8501>
Network URL: <http://172.31.76.195:8501>

app.py 11:23 LF UTF-8 4 spaces Python 3.9 (ml_semester_assignme

→ **How to Run Locally**

- **Install dependencies:**

- #bash pip install -r requirements.txt

- **Train models and generate files:**

- #python train_models.py

- **Run the Streamlit app:**

- #bash streamlit run app.py